DBMS LAB · ASSIGNMENT 4

Relational Databases and NoSQL Databases: A Comparative Report

How the two approaches differ, why organizations choose one or both, and what it means for an airline reservation system

STUDENT
Reddy Sai Abhiram

GITHUB
github.com/reddysaiabhiram2468

PROJECT
Airline Reservation & Flight Booking System

DATABASE
MySQL 8.0 · `airline_reservation`

SUBMITTED
25 October 2026

1. Background

The relational model was proposed by E. F. Codd at IBM in 1970. It describes data as relations (tables) and lets users ask for *what* they want instead of *how* to find it. That idea became SQL and a generation of products, including Oracle, IBM Db2, SQL Server, MySQL and PostgreSQL, which have been the default storage for business software ever since.

In the 2000s, large web companies needed to store more data, serve more users and change their data formats faster than a single relational server allowed. Google described Bigtable (2006) and Amazon described Dynamo (2007). Open-source systems followed: Apache Cassandra (first built at Facebook for inbox search), MongoDB, Redis, HBase and Neo4j. The label **NoSQL** became popular around 2009 and is now usually read as “not only SQL”: these systems are alternatives for particular workloads, not replacements for SQL everywhere.

2. How Each Model Stores the Same Booking

A round-trip booking for two travellers shows the difference clearly.

Relational: normalized rows in separate tables

```
-- Relational (airline_reservation, 5 tables): each fact stored once
bookings       (booking_id 7, passenger_id 3, booking_reference 'K7QF2M', status 'Confirmed')
ticket_details (21, booking 7, flight 14, passenger 3, 'EC', '14C', 6210.00, 'Boarded')
ticket_details (22, booking 7, flight 14, passenger 4, 'EC', '14D', 6210.00, 'Boarded')
ticket_details (23, booking 7, flight 39, passenger 3, 'EC', '21A', 5880.00, 'Booked') ...
flights        (14, route 1, aircraft 1, '2026-10-02 06:10', ...)   routes (1, 'SW101', 'HYD', 'DEL', ...)
payments       (31, booking 7, 24180.00, 'UPI', 'Success', 'TXN...')
```

Document: the whole booking (PNR) in one record

```
// Document store (e.g. MongoDB): the whole PNR as one document
{ "_id": "K7QF2M", "status": "Confirmed",
  "travellers": [ { "name": "Srinivas Reddy", "passport": "M4821937" },
                  { "name": "Lakshmi Reddy",  "passport": "N1937285" } ],
  "segments": [ { "flight": "SW101", "date": "2026-10-02", "from": "HYD", "to": "DEL",
                  "seats": ["14C", "14D"], "fare": 6210 },
                { "flight": "SW102", "date": "2026-10-09", "from": "DEL", "to": "HYD",
                  "seats": ["21A", "21B"], "fare": 5880 } ],
  "payment": { "amount": 24180, "method": "UPI", "status": "Success" } }
```

The document is convenient: an itinerary page or a mobile boarding pass is one read away. But the flight's airports and times are now copied into every booking on that flight, and the seat map of SW101 on 2 October is scattered across many documents. A question such as “which seats on this flight are still free?” needs every booking

document for the flight, and nothing prevents two documents from claiming seat 14C. The relational form needs joins to rebuild an itinerary, but a single UNIQUE constraint guarantees that no seat is sold twice.

NoSQL family	Unit of storage	Representative systems	Typical workload
Key–value	A value looked up by its key	Redis, Amazon DynamoDB, etcd	Caches, sessions, rate limits, counters
Document	JSON-like documents with nesting	MongoDB, Couchbase, Firestore	Catalogues, profiles, content management
Wide-column	Partitioned rows with many sorted columns	Cassandra, HBase, Bigtable, ScyllaDB	Event logs, time series, sensor and activity data
Graph	Nodes and relationships	Neo4j, Amazon Neptune	Recommendations, networks, fraud and dependency analysis

3. Comparison on Four Criteria

3.1 Scalability

Relational. The usual path is *scale-up*: a larger machine, plus read replicas that copy the data to extra servers for read-only queries. When writes outgrow one machine, the data must be *sharded*, meaning split by a key across several databases. Joins and transactions across shards then become the application's problem. Pinterest, for example, has described how it scaled by sharding MySQL itself. Distributed SQL databases such as Google Spanner, CockroachDB and TiDB now shard automatically while keeping SQL.

NoSQL. Most NoSQL systems *scale out* by design. The partition key decides which node holds a record, nodes can be added while the system runs, and data is rebalanced automatically. The price is that efficient queries must follow the partition key, so the data model is designed around the queries rather than around the entities.

3.2 Consistency

A relational DBMS gives **ACID** transactions: a booking that inserts tickets and a payment either happens completely or not at all, other users never see half of it, and constraints such as foreign keys are checked on every change. Distributed NoSQL systems keep several replicas. Under the **CAP** trade-off, during a network partition they must choose between staying available, possibly returning stale data, and staying consistent by refusing some requests. Many choose availability and **eventual consistency** (the BASE model), letting replicas converge after a short delay. This is often tunable: Cassandra lets each query choose how many replicas must agree, and MongoDB has offered multi-document ACID transactions since version 4.0. Rules that relate two entities, such as “a ticket must belong to an existing flight”, are still enforced by the database only in relational systems.

3.3 Performance

Situation	Faster choice	Reason
Read one aggregate by its key (a user profile, a cart, a book page)	Key–value / document	One lookup, no joins; data stored together on disk
Very high write rate (logs, clicks, sensor readings)	Wide-column	Writes are appended sequentially and spread over many nodes
Ad-hoc reports joining many entities	Relational	The SQL optimizer chooses join order and indexes for any query
Multi-step updates that must be all-or-nothing	Relational	Built-in transactions and locking
Traversing relationships several levels deep	Graph	Relationships are stored as direct links, not computed by joins
Sub-millisecond repeated reads	In-memory key–value (Redis)	Data served from RAM, used as a cache in front of SQL

3.4 Use cases

Relational databases suit systems of record: banking, accounting, ERP, inventory, reservations, student and hospital records, and airline seat inventory. These systems have structured data, many relationships, strict correctness requirements and varied reporting needs. NoSQL databases suit very large or fast-changing data whose access patterns are known in advance: activity feeds, chat history, IoT telemetry, product catalogues with varying attributes, caching, search, and recommendation graphs.

3.5 Side-by-side summary

Aspect	Relational DBMS	NoSQL
Schema	Defined before data is stored; enforced	Flexible; each record may differ
Data integrity	PK, FK, UNIQUE, CHECK and triggers in the DBMS	Mostly validated by application code
Transactions	Multi-row, multi-table ACID	Often single-record; multi-record support varies
Scaling	Scale-up and replicas; sharding is extra work	Scale-out across commodity nodes
Consistency	Strong by default	Eventual or tunable
Query power	Joins, subqueries, aggregation, window functions	Simple key or document queries; limited joins
Redundancy	Minimized by normalization	Accepted to make reads cheap
Best fit	Systems of record and reporting	High volume, flexible shape, simple access paths

4. Why Organizations Choose Different Technologies

Choosing a database is an engineering trade-off. Teams usually decide by asking these questions:

1. **What happens if two users see different values for a moment?** If the answer is “money is lost” or “a seat is sold twice”, strong consistency is required.
2. **How will the data be read?** If the questions are not known in advance, SQL is safer. If every read is “get item by key”, a key–value or document store is simpler and faster.
3. **How big will it get, and how fast does it change?** Billions of small writes per day point towards wide-column stores. Thousands of transactions per day fit any RDBMS.
4. **How stable is the structure?** Products with constantly changing attributes favour flexible documents. Accounting data favours a fixed schema.
5. **What does the team know, and what will it cost?** Skills, licences, managed cloud offerings and existing reporting tools often matter as much as raw performance.

Because different parts of one product give different answers, large systems combine several stores, an approach called **polyglot persistence**.

5. Real-World Examples

Organization	What they use	Why
Google (AdWords / Ads)	Sharded MySQL → F1 on Spanner	Google's F1 paper (2013) describes moving the advertising backend from manually sharded MySQL to F1, a distributed SQL system on Spanner. It wanted horizontal scale without giving up SQL and transactions for billing-critical data.
Facebook	Created Apache Cassandra	Cassandra was built for inbox search, which needed very high write throughput across many servers. It was later released as open source and is now used widely for write-heavy data.
Instagram	PostgreSQL and Cassandra	User and media metadata stayed in sharded PostgreSQL, while high-volume data such as feeds and activity moved to Cassandra.

Organization	What they use	Why
Pinterest	Sharded MySQL	Pinterest chose to scale a well-understood relational database by sharding rather than adopt a newer NoSQL system, valuing maturity and predictable operations.
Stack Overflow	Microsoft SQL Server + Redis	A relational database is the system of record, and Redis caches hot data so that very popular pages are served from memory.
Airline reservation systems (SABRE and its successors)	Transaction processing; today relational stores plus caches	SABRE, built by American Airlines and IBM in the 1960s, was one of the first large online transaction systems. Its software evolved into IBM's Transaction Processing Facility, designed for huge numbers of short, consistent booking transactions. Selling a seat is still a strongly consistent operation, while fare search results are commonly served from caches.
Banks and stock exchanges	Relational (Oracle, Db2, PostgreSQL)	Every debit needs a matching credit; ACID transactions and audit trails are mandatory.

These summaries are based on the organizations' published engineering papers and blogs and describe the general architecture at the time of publication.

6. What This Means for the Airline Reservation System

The core of an airline reservation system is a textbook relational workload. A seat can be sold only once, a payment must belong to a real booking, and cancelling a flight must cancel every ticket on it, all inside transactions. The data is highly related (passengers, bookings, tickets, flights, routes, aircraft), and reports such as load factor per route or revenue per class need flexible joins. MySQL with the constraints and triggers of Assignment 6 is therefore the right system of record. Around that core, a large airline would add specialized stores:

Feature	Need	Possible store
Bookings, tickets, seat inventory, payments	Strong consistency, no double-selling	**MySQL** (kept)
Flight search results and fare quotes	Millions of read-only searches, short-lived results	Redis or another cache, refreshed from MySQL
Flight status and gate notifications	Very many small events pushed to many clients	Message queue + key–value store
Mobile itinerary / boarding pass view	Read the whole PNR in one request	Document store copy of each booking
Aircraft sensor and telemetry data	Huge write volume, time-ordered	Wide-column store (Cassandra) or time-series DB
Route network and connection planning	Graph traversal (A → B via C)	Graph database or in-memory graph

7. Conclusion

Relational databases offer a fixed, enforced schema, rich SQL queries and ACID transactions, which makes them the natural choice when data is structured, related and must be correct. NoSQL databases give up some of these guarantees for horizontal scale, flexible structure and very fast key-based access, which suits huge or rapidly changing data with predictable access paths. Real organizations choose by consistency needs, query patterns, scale, schema stability and cost, and they frequently combine the two. For an airline reservation system, a relational database is the correct foundation for bookings, seats and payments, and NoSQL components are useful for specialized features such as fare-search caching, notifications and telemetry.